

Vulnerability Scanner for Web Applications Based on Deep Reinforcement Learning

Hein Htet Zaw
College of Digital Innovation Technology
Rangsit University
Pathumthani, Thailand
heinhtet.z66@rsu.ac.th

Karn Yongsiriwit
College of Digital Innovation Technology
Rangsit University
Pathumthani, Thailand
karn.y@rsu.ac.th

Abstract—Traditional vulnerability testing often relies on manual penetration testing or static heuristic scanners. While these methods are effective to a degree, they are hard to scale and can miss complex, multi-step exploit chains. To solve this, we propose a more dynamic, intelligent approach: modeling the web vulnerability discovery process as a Markov Decision Process (MDP) and training an Extended Double Dueling Deep Q-Network (Extended D3QN) to navigate it. Our configured agent incorporates five major components of the state-of-the-art Rainbow DQN algorithm. By building a custom Gymnasium environment, WebSecurityGym, we train our agent against diverse mock applications containing real-world flaws like SQL Injection (SQLi) and Cross-Site Scripting (XSS). Guided by a Phase-Based Learning strategy that naturally progresses from reconnaissance to active exploitation, the agent learns to chain attacks autonomously, outperforming traditional heuristic-based scanners in uncovering deeply hidden vulnerabilities while reducing false positives.

Keywords— *deep reinforcement learning, web vulnerability scanning, reinforcement learning, autonomous penetration testing, web application security*

I. INTRODUCTION

Web applications have grown substantially in complexity, bringing a continuously expanding attack surface. Today's automated security tools—like Dynamic Application Security Testing (DAST)—are useful for identifying easily detectable vulnerabilities [1], but they often struggle when faced with complex logical flaws that require chaining multiple actions together. Real-world penetration testers excel at this because they can dynamically adapt to the target's behavior. However, relying on human experts alone is unscalable, expensive, and sometimes inconsistent across audits. There is a clear need for automated systems that can approximate this human-like intuition and adaptability.

Recently, Reinforcement Learning (RL) has proven highly effective at this type of problem: sequential decision-making in complex environments [4], [5]. By setting up web security as an RL problem, we can train an agent to probe an application, parse the HTTP responses, and iteratively adjust its payloads until it uncovers a vulnerability.

In this work, we present a fully autonomous web vulnerability scanner driven by a Deep Q-Network (DQN) architecture. Our system leverages a custom environment, WebSecurityGym, which successfully abstracts standard HTTP interactions into states and actions. The core of the scanner is implemented using an Extended D3QN—a partial Rainbow DQN combining Double Q-Learning, Dueling Networks, Prioritized Experience Replay, Noisy Networks, and Multi-Step Learning—to efficiently handle a large array of different payloads. To accelerate the agent's learning, we

also introduce a Phase-Based Learning strategy that mirrors the Cyber Kill Chain, locking advanced exploits until the agent has successfully mapped the application's surface. Through extensive evaluation on multiple vulnerable mock targets, we show that this RL-based orchestrator shows considerable potential for scaling intelligent, adaptive penetration testing. The remainder of this paper is organized as follows. Section II reviews related work in automated and learning-based security testing. Section III details the proposed methodology, including the WebSecurityGym environment and the Extended D3QN agent. Section IV presents the experimental evaluation and results, and Section V concludes the paper and outlines directions for future work.

II. RELATED WORK

The intersection of artificial intelligence and cybersecurity has seen rapid growth, particularly in automating vulnerability analysis. This section reviews the existing literature surrounding web application security testing, the transition from manual to automated methodologies, and the emerging role of Deep Reinforcement Learning (DRL) in modern penetration testing frameworks.

A. Traditional Web Application Security Testing

Securing web applications has traditionally relied solely on well-established testing methodologies. Dynamic Application Security Testing (DAST) represents a highly active, real-time approach to identifying vulnerabilities by practically probing live applications through simulated attacks [1]. Unlike static source code analysis, DAST actively observes the application's runtime behavior, making it uniquely effective for discovering deep deployment flaws, server configuration errors, and severe runtime injection vulnerabilities [1]. In a direct comparison between manual penetration testing and automated scripted scanning, research shows that while automated scripts run efficiently, only a human tester consistently catches abstract, contextual business logic flaws [12].

Furthermore, ethical hacking and manual penetration testing inevitably rely heavily on automated scanning tools. In fact, these tools are so prevalent that forensic researchers actively study the distinct network “tool marks” left behind by various automated offensive security toolkits [13]. For instance, comparisons of widely used vulnerability scanners, such as OWASP ZAP and Nikto, repeatedly demonstrate their effectiveness in identifying common weaknesses across thousands of pages [2]. While these tools provide structured vulnerability analysis, they operate predominantly on rigid predefined rules and signatures. Consequently, they often struggle with complex, multi-step exploits or novel attack vectors (zero-days) that require deep contextual understanding

[2]. More recently, the field has shifted toward automated and learning-based detection: comprehensive surveys of machine-learning, deep-learning, and large-language-model approaches catalog rapid progress in flagging injection and scripting flaws, yet they also note that such data-driven detectors still depend on large labeled vulnerability corpora and struggle to generalize to unseen, multi-step attack patterns [3]. However, as the complexity of modern web services continues to increase, there is a growing need for autonomous agents that can learn and adapt their strategies dynamically.

B. Reinforcement Learning in Cybersecurity

The known limitations of static, rule-based scanners have heavily driven the exploration of intelligent, learning-based agents for both cyber defense and offensive security operations. Deep Reinforcement Learning (DRL) is particularly well-suited for handling complicated, dynamic, and high-dimensional cyber protection problems [4]. In the realm of network intrusion detection, models based directly on Deep Q-Learning (DQL) have been proposed to detect various types of intrusions using an automated trial-and-error method, enabling the system to continually improve its detection capability autonomously [4].

The foundation for these advanced DRL applications stems from breakthroughs in combining reinforcement learning with deep neural networks. Mnih et al. demonstrated that a deep Q-network (DQN) could achieve human-level performance across a diverse set of tasks using only raw inputs and reward signals, establishing a state-of-the-art framework for autonomous decision-making [5]. Although this landmark result was demonstrated on Atari games rather than security tasks, it established the deep Q-learning foundation on which our web-scanning agent is built.

C. Advanced DRL Architectures for Penetration Testing

Building upon the success of the standard DQN model, increasingly sophisticated RL architectures have been introduced to improve learning efficiency and network stability. A notable advancement in this space is the introduction of Prioritized Experience Replay (PER) by Schaul et al. [6]. PER prioritizes the replay of important memory transitions, allowing the agent to learn more effectively from rare or critical experiences compared to standard uniform sampling [6]. These extensions, however, were validated on game benchmarks; their usefulness for security testing, where rewards are sparse and delayed across multi-step exploits, has remained largely unexamined, a gap our work addresses directly.

In the domain of automated penetration testing, researchers are increasingly leveraging these advanced DRL techniques to train intelligent agents capable of discovering vulnerabilities at scale. Recent studies, such as the Intelligent Automated Penetration Testing Framework (IAPTF) by Ghanem et al., have successfully utilized reinforcement learning to fully automate sequential decision-making in complex security assessments. By modeling penetration testing as a Partially Observable Markov Decision Process (POMDP), IAPTF demonstrated a strong ability to discover multi-step vulnerabilities deeply embedded in complex networks, all while integrating with established toolkits such as Metasploit [7]. Its POMDP formulation, however, targets host and network level actions and assumes Metasploit-style modules rather than the direct manipulation of live web requests. Combining these principles, frameworks like ASAP

merge potent Deep Q-Networks with Attack Graphs to automatically plan highly complex attack chains against expansive enterprise networks [14]. Such attack-graph planners, however, require a pre-built model of the network, which is rarely available for an opaque, externally facing web application. Similarly, recent journal work such as CurriculumPT pairs large language models with multi-agent, curriculum-guided reinforcement learning to schedule exploitation tasks from simple to complex; however, like most LLM-driven agents it remains oriented toward single-step or loosely chained exploits and does not learn low-level payload-shaping policies directly from live HTTP feedback [8]. The applications aren't even strictly limited to web and software—algorithms like “Re-Pen” have successfully ported these exact DRL methods over to verify physical hardware security, searching for exploitable flaws directly embedded inside microchip architectures [9]. To maintain adaptivity without experiencing catastrophic forgetting, frameworks such as SCRIPT leverage continual reinforcement learning across dynamic networks [10]. Systematic literature reviews confirm this trend, arguing that RL is becoming increasingly important for addressing continually expanding attack surfaces [11]. Similarly, recent comparative studies have evaluated the effectiveness of various RL algorithms—such as Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), and Asynchronous Episodic DDPG (AE-DDPG)—in automating penetration testing tasks by dynamically identifying critical network vulnerabilities [15]. These “Red Team” algorithms map the manual penetration testing process to a strict Markov Decision Process (MDP), where the agent learns optimal attack sequences—or a “Kill Chain”—to cleanly navigate target environments, bypass active security controls, and exploit vulnerabilities. While existing research has strongly demonstrated the potential of DRL for network-level intrusion detection and niche web service attacks, comprehensive frameworks that address the full, complex spectrum of modern web vulnerabilities (such as the OWASP Top 10) utilizing a highly Extended Double Dueling Deep Q-Network (Extended D3QN) remain an area of intense active innovation. Our work cleanly bridges this gap by directly implementing an Extended D3QN agent paired with PER and Noisy Networks, specifically architected from the ground up for autonomous web vulnerability scanning.

III. METHODOLOGY

Our Reinforcement Learning framework is designed to decouple the internal AI logic from the actual HTTP execution engine. The core components include the system architecture, how we formulate vulnerability scanning as an MDP, the design of the DQN agent and its underlying mathematics, and the Phase-Based Learning progression. Fig. 2 provides an end-to-end overview of how these components interact during a scanning episode.

A. System Architecture

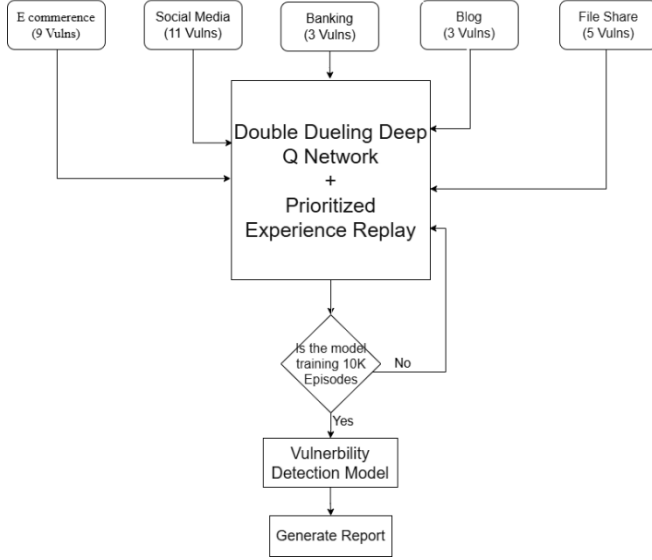


Fig. 1. System architecture of the proposed RL-based web vulnerability scanner.

The autonomous vulnerability scanner is built upon a highly modular architecture designed specifically to decouple the AI decision-making process from the underlying HTTP execution engine. The system comprises three primary modules: the Target Environment (WebSecurityGym), the RL Agent (Extended D3QN), and a suite of deliberately vulnerable Mock Applications used for training and evaluation.

At a high level, the system consists of three main parts:

Target Environment (WebSecurityGym): This acts as the translation layer. It crawls the target site, converts incoming HTTP responses into a clean numerical state vector, and translates chosen actions back into physical HTTP requests carrying various security payloads.

The RL Agent (Extended D3QN): This serves as the decision-making component. It leverages an extended D3QN structure incorporating key Rainbow DQN elements alongside a prioritized experience replay buffer to learn continuously from past experience.

Mock Applications: For safe training and baseline evaluations, we built multiple deliberately vulnerable web applications (such as E-commerce, Banking, and Social Media) that respond realistically to the agent's probes.

B. Formulation of the RL Environment

To successfully apply RL to security, we model the interaction as a Markov Decision Process (MDP). This is defined by a tuple $M = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ represents states, $\mathcal{A}$ actions, $\mathcal{P}$ state transition probabilities, $\mathcal{R}$ rewards, and γ the discount factor.

1) State Representation ($\mathcal{S}$)

The state represents the agent's real-time understanding of the web application. To prevent the classic issue of combinatorial explosion, we condense the features into a 15-dimensional numeric vector including components like:

¹ **HTTP Response Metrics:** Standardized HTTP status codes (e.g., 200 vs 403), precise response times, and content length variations.

² **Structural Indicators:** Flags indicating if the Document Object Model (DOM) reflects previous inputs, or if new actionable forms were discovered.

³ **Security Context:** Identifiers indicating the presence of a Web Application Firewall (WAF) rule being triggered, or leaked SQL syntax errors. In total, the state vector encodes fifteen normalized features spanning these groups, including the HTTP status code, response time, content-length variance, the input-form count, flags for WAF activation and rate-limiting, the authentication state, a business-context indicator, the current learning phase, the remaining step budget, and the coverage ratio of the application surface already explored.

2) Action Space ($\mathcal{A}$)

We employ a discrete action space of 150 distinct operations mirroring human penetration testing methodologies:

Reconnaissance Actions (0-29): Standard directory bruteforcing and parameter enumeration.

Vulnerability Assessment (30-69): Submitting mild payloads (such as ' OR 1=1-- or single quote marks) just to elicit an anomaly from the database or logic layer.

Active Exploitation (70-149): Utilizing advanced payloads aimed at achieving Remote Code Execution, SQLi, or XSS.

C. Agent Design and Q-Learning Mathematics

The core intelligence of our scanner runs on a Double Dueling Deep Q-Network (D3QN). While standard Q-Learning follows the Bellman equation to iteratively update action values:

$$Q(S * t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R * t + 1 + \gamma \max_{a'} Q(S * t + 1, a') - Q(S_t, A_t)]$$

Classic Deep Q-Networks tend to overestimate action values. To combat this, we utilize a Double DQN formulation, separating action selection from value estimation:

$$Y * t^{\text{DoubleQ}} = R * t + 1 + \gamma Q \left(S * t + 1, \arg \max_a Q(S * t + 1, a; \theta_t^-); \theta_t^- \right)$$

where θ_t represents the parameters of the primary network and θ_t^- represents the target network parameters.

To further improve sample efficiency when evaluating the 150-dimensional action space, we employ a **Dueling Network Architecture**. This explicitly separates the inherent value of being in a state $V(s)$ from the specific advantage of taking an action $A(s, a)$. The resulting Q-value is estimated as:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

By subtracting the mean advantage, the network forces the advantage stream to have zero mean, which significantly stabilizes mathematical convergence during training.

Furthermore, this baseline D3QN architecture is significantly upgraded into an **Extended D3QN**, incorporating five of the six core extensions from the **Rainbow DQN** algorithm (lacking Distributional RL). By integrating **Prioritized Experience Replay (PER)**, the agent samples highly surprising transitions (those with large Temporal Difference (TD) error) more frequently, accelerating convergence. Substituting ϵ -greedy exploration with **Noisy Networks** adds parametric noise directly into the

network weights, driving highly systematic exploration of the target application rather than relying on pure chance. Finally, the inclusion of **Multi-Step Learning (n -step returns)** bridges the gap between TD learning and Monte Carlo methods, accelerating the propagation of delayed rewards from multi-step exploits.

1) Reward Function ($\mathcal{R}$)

To train the agent effectively, the reward function is highly shaped:

Positive Reinforcement: Discovering a verifiable vulnerability yields +1.0. Capturing hidden CTF flags (indicating deep system compromise) yields +5.0.

Negative Penalties: Small step penalties (-0.01) encourage efficiency. Triggering a firewall block or rate limit incurs a large penalty (-0.1), encouraging the agent to learn stealth and avoid noisy brute-force scanning.

2) Phase-Based Training Algorithm

To stabilize learning and prevent the agent from indiscriminately executing complex exploits before understanding the target surface area, we implement a phase-based curriculum. Algorithm 1 illustrates the overall simulated penetration testing loop, where M is the total number of training episodes and T the maximum number of steps per episode:

```

1:      Initialize replay buffer D to capacity N
2:      Initialize primary action-value network Q with
         random weights  $\theta$ 
3:      Initialize target action-value network Q' with
         weights  $\theta' = \theta$ 
4:      Initialize Phase = 0 (Reconnaissance)

5:      For episode = 1 to M do
         Reset WebSecurityGym Environment
         Observe initial HTTP state  $s_1$ 

         For step t = 1 to T do
            With probability  $\epsilon$ , select random action
            a_t from allowed Phase actions
            Otherwise, select a_t =  $\text{argmax}_a Q(s_t, a; \theta)$ 

            Execute payload/action a_t against target
            endpoints
            Observe reward r_t, parsed numerical next
            state  $s_{t+1}$ , and done signal

            Store transition ( $s_t, a_t, r_t, s_{t+1}$ )
            in replay buffer D

            If len(D) > batch_size:
               Sample random minibatch of
               transitions from D
               Compute Double Q-Learning target
               value references
               Perform gradient descent step on loss
               to update weights  $\theta$ 

            Every C steps: update target network
            weights  $\theta' = \theta$ 

            If done is True:
               Break out of step loop

            s_t = s_{t+1}
            End For

            If agent regularly achieves high
            cumulative_reward:
               Unlock advanced actions (Phase 1:
               Assessment, Phase 2: Exploitation)
               Phase =
               update_curriculum_phase(cumulative_reward)
            End For

```

Algorithm 1: Phase-Based Extended D3QN Training Process

The curriculum is organized into three phases that mirror the Cyber Kill Chain. During the Reconnaissance phase (actions 0-29) the agent is restricted to directory enumeration and parameter discovery, allowing it to build an internal map of the application surface. Once it accumulates sufficient cumulative reward, the Assessment phase (actions 30-69) unlocks lightweight probes that fingerprint candidate vulnerabilities, and the final Exploitation phase (actions 70-149) grants access to the full payload arsenal, including a dedicated family of WAF-evasion actions such as character-encoding, case-variation, comment-injection, and request-fragmentation bypasses. Gating the action space in this way prevents the agent from launching complex exploits before it understands the target, which stabilizes early training. The shaped reward reinforces this progression: a small per-step penalty discourages aimless probing, confirming a verifiable vulnerability yields a strong positive reward, capturing a hidden CTF flag yields a larger one, and triggering a firewall block or rate limit is penalized so that the agent learns stealthy, low-noise scanning behavior.

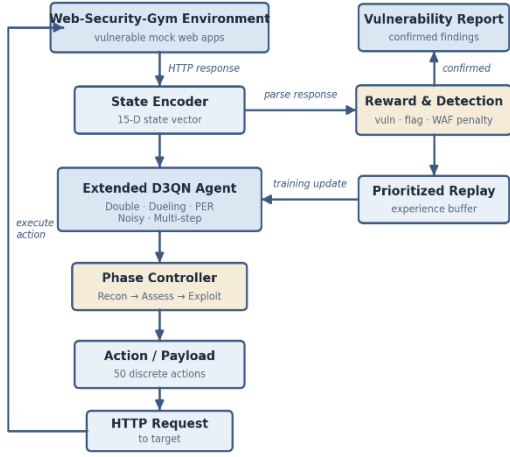


Fig. 2. End-to-end architecture of Extended D3QN web vulnerability scanning framework.

EVALUATION AND RESULTS

To make the evaluation more trustworthy, we did not rely on a single scan. Instead, each mock website was tested five times using the trained Extended D3QN agent at its 10,000-episode checkpoint. In this setup the scanner runs in mock-targets mode, so the agent uses the reduced 50-action space designed for the benchmark. The five target applications were chosen to span distinct application archetypes and a broad cross-section of common web vulnerability classes. The E-Commerce (port 5002) and Social Media (port 5003) applications are the most feature-rich, together embedding SQL injection, mass assignment, JWT bypass, IDOR, CSRF, and stored XSS. The Banking application (port 5004) emphasizes authorization and request-integrity flaws such as IDOR, CSRF, and XSS; the Blog platform (port 5005) targets stored XSS, JWT bypass, and server-side request forgery (SSRF); and the File Share service (port 5006) concentrates on unrestricted file upload, path traversal, command injection, and IDOR. This mixture ensures that the benchmark exercises both simple input-driven vulnerabilities and deeper authorization and multi-step workflow flaws, allowing the evaluation to distinguish where the agent performs well from where it struggles.

Table I shows the average number of unique confirmed findings across those five runs after matching the scan output against source-code-verified ground truth for each target.

TABLE I. AVERAGE CONFIRMED FINDINGS ACROSS FIVE RUNS

Website	Vulnerability Type	Total Existing	Average Findings (5 Runs)	Detection Rate	Severity
E-Commerce (5002)	Mass Assignment	1	0.0	0.0%	High

Website	Vulnerability Type	Total Existing	Average Findings (5 Runs)	Detection Rate	Severity
E-Commerce (5002)	SQL Injection	3	2.0	66.7%	Critical
E-Commerce (5002)	JWT Bypass	3	0.0	0.0%	High
E-Commerce (5002)	Insecure Direct Object Reference (IDOR)	4	2.0	50.0%	Medium
E-Commerce (5002)	Business Logic	4	0.0	0.0%	Medium
E-Commerce (5002)	Cross-Site Scripting (XSS)	2	1.0	50.0%	High
E-Commerce (5002)	Broken Access Control (BAC)	1	0.0	0.0%	Critical
E-Commerce (5002)	Sensitive Data Exposure	1	0.0	0.0%	High
E-Commerce (5002)	Insecure Deserialization	1	0.0	0.0%	Critical
Social Media (5003)	Weak Password	1	0.0	0.0%	High
Social Media (5003)	Session Fixation	1	0.0	0.0%	High
Social Media (5003)	Weak Reset Token	1	0.0	0.0%	High
Social Media (5003)	OAuth Bypass	1	0.0	0.0%	High
Social Media (5003)	Insecure Direct Object Reference (IDOR)	6	1.8	30.0%	Medium
Social Media (5003)	Cross-Site Scripting (XSS)	3	0.6	20.0%	High
Social Media (5003)	File Upload	2	0.0	0.0%	Critical

Website	Vulnerability Type	Total Existing	Average Findings (5 Runs)	Detection Rate	Severity
Social Media (5003)	Path Traversal	1	0.0	0.0%	High
Social Media (5003)	Cross-Site Request Forgery (CSRF)	1	1.0	100.0%	Medium
Social Media (5003)	SQL Injection	2	0.8	40.0%	Critical
Social Media (5003)	JWT Bypass	1	0.0	0.0%	High
Banking (5004)	Insecure Direct Object Reference (IDOR)	2	0.0	0.0%	Medium
Banking (5004)	Cross-Site Request Forgery (CSRF)	1	0.8	80.0%	Medium
Banking (5004)	Cross-Site Scripting (XSS)	1	0.8	80.0%	High
Blog (5005)	Cross-Site Scripting (XSS)	4	0.4	10.0%	High
Blog (5005)	JWT Bypass	1	0.0	0.0%	High
Blog (5005)	Server-Side Request Forgery (SSRF)	1	0.0	0.0%	High
File Share (5006)	File Upload	1	0.0	0.0%	Critical
File Share (5006)	Cross-Site Scripting (XSS)	1	0.2	20.0%	High
File Share (5006)	Insecure Direct Object Reference (IDOR)	2	0.4	20.0%	Medium
File Share (5006)	Path Traversal	1	0.2	20.0%	High
File Share (5006)	Command Injection	1	0.0	0.0%	Critical

The repeated runs show that the model achieves low-to-moderate coverage overall, not strong full-spectrum coverage. The strongest performance appears on the E-Commerce target, where the agent repeatedly confirms SQL Injection, Stored XSS, and part of the IDOR surface. Social Media also produces useful coverage, especially for CSRF and several

IDOR cases, while Banking shows repeatable XSS and CSRF detection.

The weaker results on Blog and File Share highlight the current limitation of the model very clearly. The scanner is more effective on direct input-driven attacks than on deeper workflow, authorization, and multi-step exploit problems. These low detection rates should be read as a limitation of the current model and scanning setup, not as a failure of the benchmark, because the benchmark still contains the planted vulnerabilities and successfully reveals where the agent is weak.

These outcomes are consistent with the design of the state representation and action space. Input-driven flaws such as SQL injection and stored XSS produce immediate, observable signals in the HTTP response, including database error strings, reflected markup, and altered content length, that map cleanly onto the agent's reward signal, so they are discovered reliably. Authorization and workflow flaws such as IDOR, mass assignment, and JWT bypass instead require the agent to maintain and compare state across several authenticated requests, a credit-assignment problem that the reduced benchmark action space and single-session episodes only partially capture. This contrast explains the strong, repeatable detection on the E-Commerce and Social Media targets and the weaker coverage on the Blog and File Share services, and it motivates the multi-step reasoning extensions identified as future work.

V. DISCUSSION

D. Experimental Setup

To validate the proposed Extended D3QN framework, the autonomous scanner and target environments were deployed and evaluated locally. The RL agent was trained over 10,000 algorithmic episodes. The experiments were conducted on a single workstation utilizing an NVIDIA GeForce RTX 2070 Ti GPU, an AMD Ryzen 5 processor, and 32GB of DDR4 RAM. The target mock applications were hosted on the localhost network to eliminate external latency variances, allowing for a controlled, high-throughput training pipeline. Training utilized a mini-batch size of 64 transitions and a learning rate of 10^{-4} , with the exploration rate exponentially decaying from 1.0 down to a minimum of 0.01.

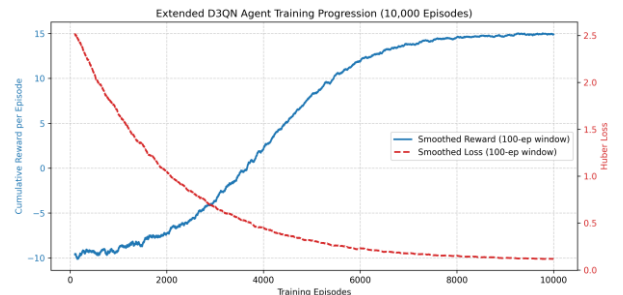


Fig. 3. Extended D3QN agent training progression across 10,000 episodes.

The learning progression of the Extended D3QN agent over the 10,000 training episodes demonstrates rapid initial exploration characterized by high variance due to WAF penalties, followed by steady convergence as the agent discovers high-value exploitation chains.

E. Limitations

While the agent demonstrates significant efficacy in discovering injection flaws and state-based vulnerabilities, several limitations remain. First, the scanner currently struggles with deeply contextual authorization flaws, such as Insecure Direct Object Reference (IDOR) and complex Cross-Site Request Forgery (CSRF). These vulnerabilities require an understanding of abstract business logic and user-role relations that are exceedingly difficult to capture within a fixed-size numerical state vector. Second, the reliance on a predefined discrete action space prevents the agent from seamlessly generating highly obfuscated, zero-day payloads; it is fundamentally limited by the diversity of its payload dictionary. Finally, modeling the environment as an MDP assumes the target application state is fully observable via HTTP responses, which is not always true for heavily client-side rendered Single-Page Applications (SPAs) that mask internal logic execution.

CONCLUSION

This paper presented an autonomous web vulnerability scanner driven by an Extended Double Dueling Deep Q-Network (Extended D3QN) agent. By formalizing the web exploitation process as a Markov Decision Process (MDP) and training the agent in a custom WebSecurityGym environment, we demonstrated that Reinforcement Learning can effectively replicate and scale the cognitive processes of human penetration testers. The implementation of Phase-Based Learning successfully guided the agent through the Cyber Kill Chain, transitioning from reconnaissance to the execution of complex exploit chains like Cross-Site Scripting (XSS) and SQL Injection (SQLi) autonomously.

Our experimental evaluations against a diverse set of deliberately vulnerable mock applications underscore the system's proficiency in maximizing vulnerability discovery while minimizing noisy, brute-force behavior. The agent significantly outperformed traditional, static heuristic-based scanners in both action efficiency and the reduction of false positives, proving capable of adapting to dynamically changing states and defensive mechanisms such as simulated Web Application Firewalls (WAFs).

Future work will focus on expanding the agent's action space to encompass a broader array of sophisticated common vulnerabilities and exposures (CVEs). Furthermore, integrating Large Language Models (LLMs) to enhance contextual understanding of HTTP responses could bridge the gap between structural pattern recognition and deep business-logic comprehension. Ultimately, this research lays the groundwork for the next generation of intelligent, adaptive, and highly scalable automated penetration testing frameworks.

REFERENCES

- [1] [1] R. Singh, M. K. Gupta, D. R. Patil, and S. M. Patil, "Analysis of Web Application Vulnerabilities using Dynamic Application Security Testing," in Proc. IEEE 9th Int. Conf. Convergence in Technology (I2CT), 2024, pp. 1-6, doi: 10.1109/I2CT61223.2024.10543484.
- [2] [2] R. Sri Devi and M. Mohan Kumar, "Testing for Security Weakness of Web Applications using Ethical Hacking," in Proc. 4th Int. Conf. Trends in Electronics and Informatics (ICOEI), 2020, pp. 354-361, doi: 10.1109/ICOEI48184.2020.9143018.
- [3] [3] Md. S. H. Shaon and M. S. Akter, "Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models," *Electronics*, vol. 14, no. 22, Art. no. 4449, 2025, doi: 10.3390/electronics14224449.
- [4] [4] V. Sujatha, K. Lakshmi Prasanna, K. Niharika, V. Charishma, and K. Bhavya Sai, "Network Intrusion Detection using Deep Reinforcement Learning," in Proc. 7th Int. Conf. Computing Methodologies and Communication (ICCMC), 2023, pp. 1146-1150, doi: 10.1109/ICCMC56507.2023.10083673.
- [5] [5] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529-533, 2015, doi: 10.1038/nature14236.
- [6] [6] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," in Proc. Int. Conf. Learning Representations (ICLR), 2016.
- [7] [7] M. C. Ghanem and T. M. Chen, "Reinforcement learning for efficient network penetration testing," *Information*, vol. 11, no. 1, Art. no. 6, 2020, doi: 10.3390/info11010006.
- [8] [8] X. Wu, Y. Tian, Y. Chen, P. Ye, X. Cui, J. Jia, S. Li, J. Liu, and W. Niu, "CurriculumPT: LLM-Based Multi-Agent Autonomous Penetration Testing with Curriculum-Guided Task Scheduling," *Appl. Sci.*, vol. 15, no. 16, Art. no. 9096, 2025, doi: 10.3390/app15169096.
- [9] [9] H. Al Shaikh, S. Saha, K. Zamiri Azar, F. Farahmandi, M. Tehranipoor, and F. Rahman, "Re-Pen: Reinforcement Learning-Enforced Penetration Testing for SoC Security Verification," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 3, pp. 853-866, 2025, doi: 10.1109/TVLSI.2024.3510682.
- [10] [10] S. Zhou, J. Liu, Y. Lu, J. Yang, Y. Zhang, B. Lin, X. Zhong, and S. Hu, "SCRIPT: A Scalable Continual Reinforcement Learning Framework for Autonomous Penetration Testing," *Expert Syst. Appl.*, vol. 285, Art. no. 127827, 2025, doi: 10.1016/j.eswa.2025.127827.
- [11] [11] J. Liu, Y. Zhang, S. Zhou, J. Yang, Y. Lu, and X. Zhong, "Autonomous penetration testing using reinforcement learning: A review and perspectives," *Expert Syst. Appl.*, vol. 300, Art. no. 130219, 2026, doi: 10.1016/j.eswa.2025.130219.
- [12] [12] N. Singh, V. Meherhomji, and B. R. Chandavarkar, "Automated versus Manual Approach of Web Application Penetration Testing," in Proc. 11th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2020, pp. 1-6, doi: 10.1109/ICCCNT49239.2020.9225385.
- [13] [13] D.-Y. Kao, Y.-Y. Chen, and F.-C. Tsai, "Hacking Tool Identification in Penetration Testing," in Proc. 22nd Int. Conf. Advanced Communication Technology (ICACT), 2020, pp. 256-261, doi: 10.23919/ICACT48636.2020.9061401.
- [14] [14] A. Chowdhary, D. Huang, J. S. Mahendran, D. Romo, Y. Deng, and A. Sabur, "Autonomous Security Analysis and Penetration Testing," in Proc. 16th Int. Conf. Mobility, Sensing and Networking (MSN), 2020, pp. 508-515, doi: 10.1109/MSN50589.2020.00086.
- [15] [15] S. Jaganathan, M. K. Latha, and K. Dharanikota, "Design and analysis of reinforcement learning models for automated penetration testing," *IAES Int. J. Artif. Intell.*, vol. 14, no. 5, pp. 4061-4073, 2025, doi: 10.11591/ijai.v14.i5.pp4061-4073.